

Archivo revisado: UserUseCasesConfig.java

Evaluación realizada por: Villarino Budip Daniela

PARTE 1. Nombres y organización de archivos

Para cada archivo fuente, verificar:

☒ **Sufijo correcto.** El código fuente usa .java y el bytecode usa .class. (2.1)

N/A **Nombres de archivo comunes.** Los makefiles se llaman GNUmakefile y el archivo que resume un directorio se llama README. (2.2)

☒ **Una sola clase o interfaz pública por archivo.** Es la primera declarada; las clases/interfases privadas asociadas pueden ir en el mismo archivo. (3.1)

☒ **Longitud del archivo.** No supera las 2000 líneas. (3)

☒ **Secciones separadas** por líneas en blanco y, opcionalmente, un comentario que identifica cada sección. (3)

☐ **Orden del archivo fuente:** (3.1)

- 1) Comentarios iniciales.
- 2) Sentencia package y sentencias import.
- 3) Declaraciones de clases e interfaces.

☐ **Comentario inicial estilo C** (/* ... */) que incluye: (3.1.1)

- Nombre de la clase e información de versión.
- Programador(es) y fecha.
- Aviso de copyright.
- Breve descripción del propósito del programa.

☒ **Sentencia package.** Es la primera línea que no es comentario; los import van después. (3.1.2)

☐ **Orden de las partes de la clase/interfaz:** (3.1.3)

- 1) Comentario de documentación de la clase (/** ... */).
- 2) Sentencia class o interface.
- 3) Comentario de implementación de la clase (/* ... */), si es necesario.
- 4) Variables de clase (static): primero public, luego protected, luego private.

- 5) Variables de instancia: primero public, luego protected, luego private.
- 6) Constructores.
- 7) Métodos, agrupados por funcionalidad (no por alcance o accesibilidad).

PARTE 2. Indentación y longitud de línea

Revisar que el código cumpla:

☒ **Unidad de indentación** de 4 espacios. (4)

N/A **Tabuladores** configurados cada 8 espacios (no 4), si se utilizan. (4)

☐ **Longitud de línea.** Ninguna línea supera los 80 caracteres. (4.1)

N/A **Ejemplos para documentación.** Líneas de 70 caracteres como máximo. (4.1)

☐ **Corte de líneas largas** según estos principios: (4.2)

- Cortar después de una coma.
- Cortar antes de un operador.
- Preferir cortes de nivel alto (fuera de paréntesis) sobre cortes de nivel bajo.
- Alinear la nueva línea con el inicio de la expresión del mismo nivel en la línea anterior.
- Si lo anterior produce código confuso o pegado al margen derecho, indentar 8 espacios.

☒ **Declaraciones de métodos largos.** Usan 8 espacios cuando la alineación convencional desplaza demasiado los parámetros. (4.2)

N/A **Condiciones de if** en varias líneas. Usan la regla de 8 espacios para que el cuerpo del bloque se distinga con claridad. (4.2)

N/A **Operador ternario** con una de las tres formas aceptadas: (4.2)

- alpha = (expr) ? beta : gamma;
- ? beta en la misma línea y : gamma alineado debajo del ?
- ? beta y : gamma cada uno en su propia línea, alineados

PARTE 3. Comentarios

Reglas generales para todos los comentarios:

- ☐ **Tipo correcto.** /* ... */ y // para comentarios de implementación; /** ... */ para comentarios de documentación (javadoc). (5)

N/A **Aportan información** que no es evidente en el código; no duplican lo que el código ya expresa claramente. (5)

N/A **Sin información** que se desactualice con facilidad ni datos ajenos al programa (cómo se construye el paquete, directorio donde reside). (5)

N/A **Decisiones de diseño** no obvias están explicadas cuando corresponde. (5)

N/A **Código confuso** se reescribe en lugar de compensarlo con más comentarios. (5)

- ☒ **Sin cajas decorativas** hechas con asteriscos u otros caracteres. (5)
- ☒ **Sin caracteres especiales** como form-feed o backspace. (5)

Comentarios de implementación:

- ☐ **De bloque.** Al inicio de cada archivo y antes de cada método; precedidos de una línea en blanco; con * al inicio de cada línea excepto la primera; indentados al nivel del código que describen. (5.1.1)

N/A **De una línea.** Indentados al nivel del código que sigue y precedidos de una línea en blanco; si no caben en una línea, usan formato de bloque. (5.1.2)

N/A **Al final de línea** (trailing). Muy cortos, separados lo suficiente de la sentencia y alineados entre sí; no se comenta cada línea al estilo ensamblador. (5.1.3)

N/A **Con //**. No se usan en varias líneas consecutivas para texto; sí se permiten para comentar bloques de código. (5.1.4)

Comentarios de documentación (doc comments):

- ☐ **Presentes en cada clase**, interfaz, constructor, método y campo, uno por API, justo antes de la declaración. (5.2)

N/A **Indentación del documento.** Clases e interfaces sin indentar (líneas siguientes con 1 espacio); miembros con 4 espacios en la primera línea y 5 en las siguientes. (5.2)

- ☐ **Etiquetas javadoc** (@param, @return, @see) usadas donde aplican. (5.2)

N/A **Detalles de implementación** fuera del doc comment. Van en un comentario de bloque o de una línea inmediatamente después de la declaración. (5.2)

- ☒ **Ningún doc comment** dentro del cuerpo de un método o constructor. (5.2)

PARTE 4. Declaraciones

Para cada declaración, verificar:

- ☑ **Una declaración por línea**, preferentemente acompañada de un comentario. (6.1)
- ☑ **Nunca variables** y métodos en la misma línea (ej. long dbaddr, getDbaddr();). (6.1)
- ☑ **Sin tipos distintos** en la misma línea (ej. int foo, fooarray[];). (6.1)

N/A **Declaraciones al inicio** de los bloques { }, no hasta el primer uso. La única excepción es el índice del for. (6.2)

N/A **Sin ocultar declaraciones**. No se redeclara en un bloque interno un nombre ya declarado en un nivel superior. (6.2)

N/A **Variables locales** inicializadas donde se declaran, salvo que el valor inicial dependa de un cálculo previo. (6.3)

- ☑ **Formato de clases, interfaces y métodos:** (6.4)
 - Sin espacio entre el nombre del método y el paréntesis “(” de sus parámetros.
 - La llave “{” al final de la misma línea de la declaración.
 - La llave “}” en su propia línea, alineada con la sentencia que abre; en métodos vacíos va inmediatamente después: {}.
 - Métodos separados por una línea en blanco.

PARTE 5. Sentencias

Para cada sentencia, verificar:

- ☑ **Máximo una sentencia** por línea (evitar argv++; argc--;). (7.1)
- ☑ **Sin operador coma** para agrupar sentencias, salvo razón evidente. (7.1)

N/A **Sentencias compuestas**. Contenido indentado un nivel más; “{” al final de la línea que las inicia; “}” al inicio de línea, alineada con la sentencia. (7.2)

N/A **Llaves siempre** en estructuras de control (if, else, for, while, do), incluso con una sola sentencia. (7.2 / 7.4)

- ☑ **return sin paréntesis**, salvo que aclaren el valor devuelto. (7.3)

N/A **if / if-else / else-if** con la forma: (7.4)

N/A **for con la forma** for (inicialización; condición; actualización) { ... }; el for vacío termina en punto y coma. (7.5)

N/A **Operador coma en el for** con tres variables como máximo; si se requieren más, usar sentencias separadas antes o al final del ciclo. (7.5)

N/A **while con la forma** while (condición) { ... }; el while vacío termina en punto y coma. (7.6)

N/A **do-while** con la forma do { ... } while (condición);. (7.7)

N/A **switch:** (7.8)

N/A **try-catch** con la forma try { ... } catch (ExceptionClass e) { ... }. (7.9)

- if (condition) { ... } else if (condition) { ... } else { ... }
- Cada case que continúa al siguiente (sin break) tiene el comentario /* falls through */ donde iría el break.
- Siempre incluye un caso default, y este termina con break.

PARTE 6. Espacios en blanco

Revisar que existan dos líneas en blanco:

- ☐ **Entre secciones** de un archivo fuente. (8.1)

N/A **Entre definiciones** de clases e interfaces. (8.1)

Revisar que exista una línea en blanco:

- ☒ **Entre métodos.** (8.1)

N/A **Entre las variables locales** de un método y su primera sentencia. (8.1)

N/A **Antes de un comentario** de bloque o de una línea. (8.1)

N/A **Entre secciones lógicas** dentro de un método. (8.1)

Revisar el uso de espacios:

N/A **Palabra clave** seguida de paréntesis separada por un espacio (while (true)). (8.2)

- ☒ **Sin espacio** entre el nombre de un método y “(”. Así se distinguen las llamadas de las palabras clave. (8.2)

- ☒ **Espacio después** de cada coma en listas de argumentos. (8.2)

N/A **Operadores binarios** (excepto el punto) separados de sus operandos por espacios (a = (a + b) / (c * d);). (8.2)

N/A **Operadores unarios** (menos unario, ++, --) sin espacio respecto de su operando. (8.2)

N/A **Expresiones del for** separadas por espacios (for (expr1; expr2; expr3)). (8.2)

N/A **Casts** seguidos de un espacio ((byte) aNum). (8.2)

PARTE 7. Convenciones de nombres

Para cada identificador, verificar:

- ☒ **Clases.** Sustantivos en mayúsculas y minúsculas, con la primera letra de cada palabra en mayúscula (ImageSprite); simples y descriptivos. (9)
- ☐ **Sin acrónimos** ni abreviaturas en nombres de clase, salvo los más usados que su forma larga (URL, HTML). (9)

N/A **Interfaces.** Misma capitalización que las clases (RasterDelegate, Storing). (9)

- ☒ **Métodos.** Verbos con la primera letra en minúscula y la primera letra de cada palabra interna en mayúscula (run(), getBackground()). (9)
- ☒ **Variables.** Primera letra en minúscula y palabras internas con mayúscula (myWidth); cortas, significativas y mnemónicas. (9)
- ☒ **Sin variables de un solo carácter**, excepto temporales: i, j, k, m, n para enteros; c, d, e para caracteres. (9)

N/A **Constantes.** Todo en mayúsculas con palabras separadas por guion bajo (MIN_WIDTH, MAX_WIDTH). (9)

PARTE 8. Prácticas de programación

Revisar que el código no contenga:

- ☒ **Variables de instancia** o de clase públicas sin una buena razón (se aceptan solo si la clase es una estructura de datos sin comportamiento). (10.1)

N/A **Acceso a miembros static** mediante un objeto (anObject.classMethod()); debe usarse el nombre de la clase. (10.2)

- ☒ **Números mágicos.** Literales numéricos codificados directamente, excepto -1, 0 y 1 como contadores de un for. (10.3)
- ☒ **Asignaciones múltiples** en una sola sentencia (a.x = b.y = 'c';). (10.4)
- ☒ **Asignaciones que se confunden** con comparaciones, por ejemplo dentro de la condición de un if. (10.4)
- ☒ **Asignaciones embebidas** para “mejorar el rendimiento” (d = (a = b + c) + r;). (10.4)

Revisar que el código sí aplique:

N/A **Paréntesis en expresiones** con operadores mezclados (if ((a == b) && (c == d))). (10.5.1)

- ☒ **Estructura que refleja** la intención. return expresión; en lugar de un if/else que devuelve true/false; ternario cuando corresponda. (10.5.2)

N/A **Paréntesis antes** del “?” cuando la expresión contiene un operador binario ((x >= 0) ? x : -x). (10.5.3)

N/A **Comentarios especiales.** XXX para algo incorrecto que funciona; FIXME para algo incorrecto y que falla. (10.5.4)

PARTE 9. Comparación contra el ejemplo de referencia

Comparar cada archivo con el ejemplo del estándar:

- ☐ **Estructura general** coincide con el orden: (11.1)
 - Comentario inicial → package → import → doc de la clase → class
 - Comentario de implementación → variables static → variables de instancia
 - Constructores → métodos
- ☐ **Doc comment de la clase** incluye descripción, @version y @author. (11.1)
- ☐ **Cada variable, constructor** y método tiene su propio doc comment. (11.1)
- ☐ **Métodos con parámetros** documentan cada uno con @param. (11.1)

RESULTADO DE LA EVALUACIÓN

Puntos por corregir:

1. Orden del archivo fuente (3.1)

Falta el comentario inicial requerido antes de package e imports; el archivo pasa directamente a package.

2. Comentario inicial estilo C (3.1.1)

No existe un comentario inicial con versión, autor, fecha, copyright y descripción del propósito.

3. Orden de las partes de la clase/interfaz (3.1.3)

Falta el doc comment de la clase y la estructura no incluye los elementos indicados antes de los métodos.

4. Longitud de línea (4.1)

Hay imports que superan los 80 caracteres, por lo que deben ajustarse al límite de la checklist.

5. Corte de líneas largas (4.2)

Las líneas que superan el límite no siguen una estrategia de corte que permita cumplir con los 80 caracteres.

6. Tipo correcto de comentarios (5)

El archivo no contiene los comentarios de implementación y documentación requeridos por la checklist.

7. Comentarios de implementación de bloque (5.1.1)

No existe el comentario de bloque inicial ni comentarios de bloque antes de los métodos.

8. Doc comments para las APIs (5.2)

La clase y los métodos no tienen comentarios Javadoc inmediatamente antes de sus declaraciones.

9. Etiquetas Javadoc (5.2)

No se utilizan las etiquetas requeridas, como `@param`, donde corresponden.

10. Dos líneas en blanco entre secciones (8.1)

Las secciones del archivo están separadas por una sola línea en blanco, no por dos.

11. Convención de nombres de clase (9)

El nombre `UserUseCasesConfig` contiene la abreviatura `Config`, que la checklist indica evitar en nombres de clase.

12. Estructura general contra el ejemplo de referencia (11.1)

No coincide completamente con el orden del ejemplo porque faltan el comentario inicial y los doc comments.

13. Doc comment de la clase con `@version` y `@author` (11.1)

`UserUseCasesConfig` no tiene el doc comment de clase ni las etiquetas `@version` y `@author`.

14. Doc comment propio para cada variable, constructor y método (11.1)

Los cuatro métodos no tienen su propio doc comment.

15. `@param` para métodos con parámetros (11.1)

Los parámetros de los métodos no están documentados con etiquetas `@param`.